

UMians EduGig: A Student Freelance Services Mobile Application



**In Partial Fulfillment of the Requirements
In CCE106/L – Applications Development and Emerging Technologies**

Submitted by:

April Bords M. Nerosa

Submitted to:

Michael Aguido Velez

September 12, 2026

Table of Contents

Chapter 1 3

 I. Introduction 3

 II. Purpose and Value Proposition 3

 III. Target Audience..... 4

 IV. Core Functionalities..... 5

 V. Technical Feasibility & Flutter Elements 9

Chapter 1

I. Introduction

Students at the University of Mindanao already hire each other for small paid jobs: a poster for an org event, a tutor before an exam, someone to proofread a thesis chapter. Today those deals happen in Facebook groups and group chats, where posts disappear within hours, nobody knows who is reliable, and nothing records what was agreed. UMians EduGig is a Flutter mobile application that gives UM students one place to offer and hire skill-based services. A student signs in with their UM Google account, publishes a service listing or browses the ones already there, places an order, chats with the other party, receives the work, and leaves a review. Every account can both buy and sell, so there is no separate freelancer registration. The app runs on Firebase (Authentication, Cloud Firestore, Cloud Storage, and Cloud Functions) and targets Android, with the same code running on the web and Windows for testing.

II. Purpose and Value Proposition

The purpose of UMians EduGig is to turn informal peer hiring into orders that both sides can trust without knowing each other first. The current process fails in four ways. Discovery is poor because posts scroll away and cannot be searched. Trust is weak because testimonials are screenshots, which are easy to fake. Structure is missing because the price, the deadline, and the number of revisions are never written down. Payment is awkward because either the buyer pays first and hopes, or the freelancer works first and hopes. Global platforms such as Upwork and Fiverr solve these for professionals, but their dollar pricing, worldwide competition, and payout requirements make no sense for a student charging 300 pesos for an hour of tutoring.

The app answers each of these gaps. Listings are searchable by category, keyword, and price. Reviews can only be written by the buyer of a completed order, one per order, so they cannot be faked or duplicated. Every order freezes its price, delivery days, revisions, and requirements at creation, and neither side can edit them afterwards. Payment goes through a Xendit checkout, and the platform holds the money until the client accepts the delivery, refunds it if the order is cancelled, or lets staff decide on a dispute. The platform keeps a 10 percent commission on completed orders and offers an optional Pro subscription for featured listings and a verified badge. What makes the app different is campus scale. The person you hire may sit beside you next semester, and the app adds a written record and a review trail on top of trust that already exists.

III. Target Audience

The users of UMians EduGig are students of the University of Mindanao, mostly college students aged 18 to 24. Only a verified umindanao.edu.ph Google account can sign in, so every user on the platform is a UM student, and the student number is read from the email address itself. Buying is open to everyone; selling and payouts require a birth date showing 18 or over. Each account can act in two roles. Buyers need short, deadline-bound tasks such as thesis proofreading, tutoring, posters for a student organization, video editing for a defense, or help with a programming assignment. They are price sensitive and already used to hiring peers informally, but they lack a quick way to find the right person and some assurance that the work will arrive. Sellers have a skill someone will pay for, such as design, programming, writing, video, photography, or music, and want income that fits between classes. They cannot commit to fixed hours and are not competitive on global platforms, but they are well matched to campus-scale work.

The app fits them by pricing in pesos, measuring delivery in days, and organizing listings into the eight categories students actually trade in: Tutoring, Graphic Design, Writing and Editing, Programming, Video Editing, Photography, Music and Audio, and Other. Profiles show each student's college and program from a closed list, so a buyer can see that a calculus tutor really is from the College of Engineering. A seller can pause a listing during exam week instead of deleting it, and released earnings are paid out to GCash, Maya, or a bank account.

IV. Core Functionalities

1. Marketplace with Search, Offers, and Ordering

The marketplace shows every published service as a card with its title, category, price, delivery time, seller, and rating. Buyers filter by category with a strip of ChoiceChip widgets, search with a TextField, and sort by newest or price. Results load through a ListView.builder on phones and a GridView on wider screens, paging with a Firestore cursor as the user scrolls. When a seller saves a listing, the app derives keyword tokens from the title and skills, so "poster" finds "Event poster for your student org". A listing is either fixed price, which can be ordered directly, or a starting price, where the buyer messages the seller and the seller sends an offer card in the chat with the agreed price, delivery days, and scope; accepting the card creates the order from those figures. This matters because a listing that cannot be found two weeks after posting means nothing else in the app gets used. Tapping a card pushes the service page through go_router, where the order or message button stays pinned at the bottom of the screen.

2. Order Lifecycle with Held Payment

An order records what was agreed and moves through pending, accepted, in progress, submitted, and completed, with a revision loop from submitted and with rejected, cancelled, and disputed as exits. On creation the price and terms are copied from the service inside a Firestore transaction, so a tampered request cannot change them. A transition table lists which state may move to which and which party may make each move: the seller accepts, starts, and submits; the buyer completes or asks for a revision. This is enforced three times. The screen only shows legal buttons, the repository re-checks inside a transaction, and Firestore security rules reject anything else. Once the seller accepts, the buyer pays through a Xendit checkout page and the platform holds the money until the order ends, then releases it to the seller's wallet minus commission, or refunds it. A client who ignores a delivery for three days does not keep the freelancer unpaid; a scheduled Cloud Function completes the order. The order screen listens to the document with a StreamBuilder, so when the other party acts, the status tracker and buttons update without a refresh. Cancelling or disputing opens an AlertDialog that asks for a reason first.

3. Chat, Reviews, and Notifications

A buyer and seller message each other from a service page or an order, with text, image, and file attachments stored in Cloud Storage. The conversation id is the two user ids sorted and joined, so a pair always lands in the same thread. A reversed ListView.builder keeps the newest message at the bottom, and the unread badge on the NavigationBar updates from the same stream. After completion the buyer leaves a star rating and comment through a dialog. The review's document id is the order id, so a second review on the same order is impossible, and the same atomic batch updates the service's rating counters. Every order and chat event writes an

inbox document for the recipient, which a Cloud Function fans out as a push notification through Firebase Cloud Messaging. Tapping a notification goes through one NotificationRouter that validates the id and opens the exact order or chat. These pieces matter because negotiation, delivery, and the evidence for a dispute all live in the chat, and reviews are the trust signal a screenshot cannot provide.

4. Admin Console with Role-Based Access

Staff run the platform from a console inside the same app. Access is the existence of an admins document for the signed-in student, written only with a service account key, and it carries a role and a list of permissions: manage students, moderate listings, manage categories, resolve disputes, settle payouts, handle refunds, decide identity verification, view reports, and edit platform settings. The screen builds one section per permission the staff member actually holds, as a NavigationRail on wide screens and a ChipStrip on phones, and each queue is a StreamBuilder over a ListView.builder with a Badge showing how many items are waiting. Disputes close as a full release or a full refund from an AlertDialog that records the reason, and orders of 2,000 pesos or more need one staff member to propose and a different one to confirm. Payouts are sent through Xendit or recorded by hand, stuck refunds are retried, chargebacks are passed on to the seller, and a student ID is approved only after the reviewer confirms it matches the declared name and birth date. Staff can also suspend an account, take down a listing, post an announcement, and pause new orders platform-wide. The same permission names are checked in the security rules and in the backend, the main admin grants staff but never themselves, and every staff action lands in an audit log that only the backend can write. This matters because money and identity decisions need a named person behind them and a record that cannot be edited afterwards.

V. Technical Feasibility & Flutter Elements

UMians EduGig is feasible with Flutter and Firebase because both are free to start, well documented, and already used in the course. The app is written in Dart 3 with Material 3 and uses `firebase_core`, `firebase_auth`, `cloud_firestore`, `firebase_storage`, and `firebase_messaging` for the backend, `go_router` for navigation, `provider` for state, `intl` for peso and date formatting, and `file_picker` and `url_launcher` for attachments and the checkout page. Code under `lib` is split by feature into `data`, `domain`, and `presentation` folders. Only repositories touch the Firebase SDKs; no widget imports Firestore. Repositories are created once at startup as plain Providers, and a single `AuthController` `ChangeNotifier` drives `go_router`'s redirect, so a signed-out or non-UM account cannot reach a protected route. A `ShellRoute` wraps the four tabs (Home, Orders, Messages, Me) in a `NavigationBar` on phones and a `NavigationRail` from 1000 pixels wide. Live data uses `StreamBuilder` with streams created once in `initState` and held, because a fresh stream on every rebuild tears down the Firestore listener. Forms use `Form` and `TextFormField` with validators kept in one file, and every Firebase error is mapped in the repository to a small failure class whose message is the only thing a screen shows.

Anything involving money or entitlements is decided on the server, never in the app, because an APK decompiles and a web build ships readable JavaScript. Cloud Functions in the `asia-southeast1` region hold the Xendit secret key, re-derive every amount from the order, and are the only writer of payment verification, wallet balances, payouts, and Pro subscriptions; Firestore rules make those fields unreachable from clients. A gateway callback is checked against a shared token in constant time and applied inside an idempotent transaction, and a scheduled job asks Xendit every thirty minutes about any checkout still pending, so a lost callback is a delay rather than a stuck order. A card chargeback after release is recorded by staff and taken from the seller's

balance, with any remainder repaid from their next releases. Disputes of 2,000 pesos or more need one staff member to propose the outcome and a different one to confirm it, and every staff action lands in an audit log only the backend can write.